Understanding Undefined Behaviour in C

In computer programming, undefined behaviour is defined as 'the result of compiling computer code which is not prescribed by the specs of the programming language in which it is written'. This article will help you understand this behaviour with the help of a few case studies.

Many of you might have come across the concept called 'undefined behaviour' in C language. During the runtime, we sometimes see strange results instead of an expected output. Let us dig deep into the 'C' ocean and see what undefined behaviour really is. What causes it and how can it be taken care of?

According to C99 standards, undefined behaviour is defined as: "Between two sequence points, an object is modified more than once, or is modified and the prior value is read other than to determine the value to be stored."

In C FAQs this behaviour is defined as: "Anything at all can happen; the standard imposes no requirements. The program may fail to compile, or it may execute incorrectly (either crashing or silently generating incorrect results), or it may fortuitously do exactly what the programmer intended."

In very simple words, an object when modified more than once between two sequence points will result in undefined behaviour. As a C programmer, understanding undefined behaviour is very important for better coding and for the program to yield a good performance, especially when it comes to embedded C coding in embedded system design.

Let us understand what a sequence point is in the C programming language with the help of an example.

Sequence points in C

According to the C99 standard: "Between the previous and next sequence point an object shall have its stored value modified at most once by the evaluation of an expression. Furthermore, the prior value shall be accessed only to determine the value to be stored."

Let us consider an example shown in code snippet-1 given below to understand the meaning of the above statement:

```
1. int a = 10;
2. a = a++ * a++;
```

In the expression shown in Line 2 above, it is clear that the value of the variable 'a' is getting modified more than once, since this variable lies between two sequence points. Here the sequence points are statement terminators, which is nothing but the semicolon (;) present at the end of the expressions—one is at the end of Line 1 and the other is at the end of Line 2.

Let us consider one more example to understand the meaning of the sequence points:

```
1. int i = 2;
2. a[i++] = i++;
```

In the expression shown in Line 2 in the code snippet-2 given above, it is clear that the value of the variable 'i' is getting modified more than once, since this 'i' lies between two sequence points. Even here, the two sequence points are the semicolons which are present at the end of the expressions or statements — one is at the end of Line 1 and the other is at the end of Line 2.

After understanding the meaning of the sequence points through examples, let us understand the meaning of undefined behaviour with the help of case studies.